LeCal: Latency-Aware Dataset Curation and Alignment for LeRobot Teleoperation

Chan Tobias Yin-Ching
The Chinese University of Hong Kong
ICCA 2026 Invited Session Draft

March 2026

Abstract

Teleoperated robot-learning datasets often log actions, proprioception, and video under a shared sample index, but that indexing does not guarantee physical synchrony. In low-cost leader-follower systems, actuator delay, middleware scheduling, and camera buffering can create persistent temporal offsets that degrade dataset quality. We present LeCal, a LeRobot-native post hoc calibration workflow and released CLI for estimating command-to-state delay, reporting confidence diagnostics, estimating camera-to-proprioception offset when local video is available, and exporting alignment artifacts. The method remains usable without `observation.leader_state` by treating the recorded action stream as the control-side surrogate. On a released SO-101 calibration dataset with 72 analyzed episodes, LeCal finds a concentrated action-to-state delay distribution with mean 4.194 frames (139.815 ms at about 30 Hz). Applying the estimated lag reduces mean action-state residual from 4.007 to 1.070, a 64.559% improvement. LeCal clearly outperforms synchronous indexing, a fixed 1-step shift, and GCC-PHAT, while remaining nearly tied with plain normalized cross-correlation on this joint-isolation dataset. A full benchmark over 388 episodes from eight public LeRobot corpora shows only a small overall edge over plain NCC, supporting LeCal primarily as a practical calibration workflow rather than a dramatic new correlator.

1 Introduction

Large robot-learning pipelines increasingly rely on teleoperated multimodal datasets. In LeRobot-style data, actions, proprioception, timestamps, and video may share an index while still referring to different physical instants. When that happens, the dataset remains structurally valid but becomes temporally inconsistent.

LeCal targets this problem with a practical post hoc calibration workflow for LeRobot-format teleoperation datasets. From recorded action and follower-state streams, it estimates episode-level command-to-state delay, reports confidence diagnostics, optionally estimates camera-to-proprioception offset when local video is available, and exports alignment artifacts for downstream use. The released CLI [3] keeps the workflow reproducible and usable even when `observation.leader_state` is absent by using the recorded action stream as the control-side surrogate.

The paper makes a narrow claim. On a released SO-101 calibration dataset with joint-isolation motions, LeCal materially improves command-state alignment relative to practical baselines and turns those estimates into reusable alignment artifacts. Camera analysis is supportive rather than central, and the public benchmark shows LeCal as a conservative workflow improvement rather than a universally superior correlator.

2 Related Work

Robot-learning infrastructure has moved toward shared formats, large corpora, and reusable pipelines. LeRobot and SO-101 make real-world teleoperation data easier to collect and reuse [1, 5], while larger efforts such as RT-1, ACT, Diffusion Policy, Open X-Embodiment, and DROID further emphasize temporally structured demonstrations [9, 10, 11, 12, 13, 14]. As these ecosystems scale, temporal semantics matter more: a sample can be syntactically valid yet causally misaligned. From a systems perspective, ROS 2 tracing and message-flow analysis show that middleware and scheduling can materially affect timing [6, 7]. Time-delay estimation itself is also well established, with GCC-PHAT as a standard reference [8]. The contribution here is therefore not a new generic correlator, but a reproducible LeRobot-oriented workflow that combines delay estimation, diagnostics, camera-offset analysis, and alignment export.

3 Problem Setting and Workflow

For each episode, let $t_k \in \mathbb{R}$ denote timestamps, $u_k \in \mathbb{R}^d$ the recorded command or action vector, and $x_k \in \mathbb{R}^d$ the follower state. The goal is to estimate an effective delay τ or integer lag ℓ such that follower motion is better explained as the delayed response to earlier commands. In continuous time, the target relationship is

$$x(t) \approx \mathcal{D}(u(t-\tau)),$$

where $\mathcal{D}(\cdot)$ represents the unknown follower-side dynamics. In practice, LeCal operates on the discrete recording and estimates episode-level lag in frames. The implemented workflow is explicit. Users first inspect a dataset to verify keys and timing quality, then estimate latency,

convert the resulting report into an alignment plan, and finally either export `delta_timestamps` or materialize an aligned parquet view. In the current package, these steps correspond to `inspect`, `estimate` or `run`, `plan`, `export-delta`, and `apply`. The alignment export supports two practical modes:

1. `system_id`: follower state is the reference timeline, and action is shifted backward to match the state it caused.

2. `policy`: the control-side surrogate is the reference timeline, and state and image streams are shifted forward accordingly.

The contribution is therefore not only lag measurement, but a reusable dataset artifact.

4 Method

4.1 Uniform Resampling and Derivative Features

The recorded timestamps are not assumed to be perfectly uniform. For a target frame rate f_{fps}, LeCal constructs a uniform grid

$$\tilde{t}_n = t_{\min} + nT_s, \qquad T_s = \frac{1}{f_{\text{fps}}},$$

and linearly interpolates both action and state onto that grid:

$$\begin{aligned} \tilde{u}_n &= \operatorname{Interp}\big(\{(t_k, u_k)\}, \tilde{t}_n\big), \\ \tilde{x}_n &= \operatorname{Interp}\big(\{(t_k, x_k)\}, \tilde{t}_n\big). \end{aligned}$$

To reduce ambiguity from long position holds, LeCal correlates velocity-like features rather than raw pose values. In the current implementation, these features are computed from Savitzky-Golay-smoothed trajectories followed by numerical differentiation:

$$\Delta\tilde{u}_n = \nabla\,\mathrm{SG}(\tilde{u})_n \cdot f_{\text{fps}}, \qquad \Delta\tilde{x}_n = \nabla\,\mathrm{SG}(\tilde{x})_n \cdot f_{\text{fps}}.$$

Here $\mathrm{SG}(\cdot)$ denotes Savitzky-Golay smoothing.

4.2 Activity Weighting

Idle segments can dominate correlation scores while carrying little information about delay. LeCal therefore uses activity weights derived from the magnitudes of the derivative signals. Let $\|\Delta\tilde{u}_n\|_2$ and $\|\Delta\tilde{x}_n\|_2$ denote the per-sample command and state activity. The implementation uses a thresholded weighting rule

$$w_n = \begin{cases} 1, & \text{if } \|\Delta\tilde{u}_n\|_2 > \theta \text{ or } \|\Delta\tilde{x}_n\|_2 > \theta, \\ w_{\text{idle}}, & \text{otherwise}, \end{cases}$$

with $w_{\text{idle}} < 1$. This suppresses uninformative plateaus without discarding them completely.

4.3 Primary Estimator: Activity-Weighted Normalized Cross-Correlation

The primary estimator is weighted normalized cross-correlation (WNCC) applied to the derivative features. For joint j and candidate lag $\ell \in \{0, \ldots, \ell_{\max}\}$,

$$C_j(\ell) = \frac{\sum_n w_n \big(\Delta\tilde{u}_n^{(j)} - \mu_{u,j}\big)\big(\Delta\tilde{x}_{n+\ell}^{(j)} - \mu_{x,j}\big)}{\sqrt{\sum_n w_n \big(\Delta\tilde{u}_n^{(j)} - \mu_{u,j}\big)^2}} \times \sqrt{\sum_n w_n \big(\Delta\tilde{x}_{n+\ell}^{(j)} - \mu_{x,j}\big)^2},$$

where $\mu_{u,j}$ and $\mu_{x,j}$ are weighted means over the valid overlap region for lag ℓ. Because the calibration dataset excites one joint at a time, LeCal aggregates per-joint correlation curves with activity-aware weights:

$$\alpha_j \propto \sqrt{\sum_n w_n (\Delta\tilde{u}_n^{(j)})^2}\sqrt{\sum_n w_n (\Delta\tilde{x}_n^{(j)})^2}, \qquad \sum_{j=1}^{d} \alpha_j = 1,$$

and forms the global objective

$$C_{\text{global}}(\ell) = \sum_{j=1}^{d} \alpha_j\, C_j(\ell).$$

The estimated lag is then

$$\ell^* = \arg\max_{\ell} C_{\text{global}}(\ell).$$

LeCal also records per-joint peak lags and a quadratic sub-frame refinement, though the main evaluation here remains in integer frames.

4.4 Secondary Estimators

LeCal also reports GCC-PHAT and a simple first-order-plus-dead-time fit as secondary consistency checks. In the present data, they are more useful as diagnostic comparators than as primary decision rules.

4.5 Confidence and Consistency Diagnostics

LeCal attaches several diagnostics to each selected lag estimate. The peak correlation is $C_{\text{global}}(\ell^*)$. The peak-to-sidelobe ratio (PSR) is implemented as a standardized peak separation. Let $\mathcal{S}$ denote the set of non-peak lags outside a small exclusion band around ℓ^*. Then

$$\mathrm{PSR} = \frac{C_{\text{global}}(\ell^*) - \mu_{\mathcal{S}}}{\sigma_{\mathcal{S}}},$$

where $\mu_{\mathcal{S}}$ and $\sigma_{\mathcal{S}}$ are the mean and standard deviation of the sidelobe values. To quantify within-episode consistency, LeCal also uses block bootstrap resampling and sliding-window lag estimation, summarized by the latency-stability metric

$$S = \operatorname{clip}\big(C_{\text{global}}(\ell^*), 0, 1\big) \cdot \frac{\mathrm{PSR}}{1 + \sigma_{\text{boot}}}.$$

LeCal also reports the standard deviation of sliding-window lag estimates, denoted σ_{win}, and disagreement among WNCC, GCC-PHAT, and FOPDT. These diagnostics are heuristic and expose unstable or weakly identifiable episodes rather than serving as external error bars.

4.6 Camera-Proprioception Offset Estimation

When local MP4 files are available, LeCal estimates camera-to-proprioception offset from visual motion. For grayscale frame I_n, the motion-energy signal is

$$m_n = \frac{1}{|\Omega|} \sum_{p \in \Omega} |I_n(p) - I_{n-1}(p)|.$$

The implementation first learns a nonnegative weighted kinematic proxy from joint motion,

$$s_n(\beta) = \sum_{j=1}^{d} \beta_j |\Delta \tilde{x}_n^{(j)}|, \qquad \beta_j \geq 0,$$

and selects the lag that yields the strongest correlation between m_n and that proxy. If this weighted proxy is too weak, LeCal falls back to a simpler speed-based proxy

$$s_n = \|\Delta \tilde{x}_n\|_2,$$

and aligns m_n against $s_{n+\ell}$. Gripper-only episodes are excluded from camera calibration because gripper motion does not meaningfully move the camera and would therefore produce misleading visual timing estimates.

4.7 Alignment Export

LeCal converts the estimated delays into dataset-level alignment artifacts. In `system_id` mode, follower state is the reference timeline and action is shifted backward. In `policy` mode, action is the reference and state and images are shifted forward. Because the current dataset does not contain `observation.leader_state`, the exported plan records that action is used as the leader-side surrogate.

5 Experimental Setup

We evaluate LeCal primarily as a calibration method. We collected 144 episodes of SO-101 calibration data overall and released the dataset on Hugging Face as `Toby0614/lecal_calibration` [2]. The command-state analysis in this paper uses the 72 episodes present in the processed local report artifact, corresponding to twelve joint-isolation motion templates with six repetitions each. These are the exact episodes reproduced by the companion local-analysis script and archived report. Among them, 12 are gripper-only trials; they remain part of the command-state analysis but are excluded from camera-offset analysis because the camera is effectively static. The main estimator uses action- and state-derived motion only; video is used only for the separate camera-offset module. Within the pipeline, joint handling is driven by per-joint motion energy from the recorded arrays, while folder names are metadata only. The processed outputs, paper figures and tables, and reproduction scripts are archived in a Zenodo companion package [4], while the CLI library itself is released on GitHub [3]. Outside this favorable local setting, we run a full benchmark over the complete accessible public splits of eight LeRobot corpora, totaling 388 episodes. To verify that the released CLI also works end to end on real external LeRobot datasets, we run a focused workflow audit on `dannyh147/so101_teleop` and `lerobot/aloha_static_screw_driver`. For each 5-episode subset, we run `estimate` and `compare`, convert the report to an alignment plan, apply the plan in both `system_id` and `policy` modes, and then re-run `estimate` and `compare` on the aligned output. These audits are interpreted separately from the 388-episode public estimator benchmark. No external ground-truth delay annotations are available, so evaluation is operational rather than oracle-based. The key question is whether the selected lag improves temporal self-consistency relative to practical baselines and whether the exported artifacts behave coherently when re-applied in a matched rerun. The primary baselines are therefore:

1. synchronous indexing, corresponding to lag = 0;
2. a fixed 1-step shift;
3. a GCC-PHAT correlation baseline.

For camera analysis, the relevant baseline is the default zero-offset assumption in a standard LeRobot workflow.

6 Results

6.1 Primary Result: Command-State Alignment on the Calibration Dataset

The strongest result is command-to-state alignment on the calibration dataset. LeCal identifies an action-to-state lag with mean 4.194 frames, median 4.0 frames, and range 4 to 10 frames, corresponding to 139.815 ms at about 30 Hz. The distribution is highly concentrated, with 65 of 72 episodes assigned to 4 frames. Applying that lag reduces mean tracking residual from 4.007 under synchronous indexing to 1.070, a 64.559% improvement. This is nearly tied with plain NCC at 1.075 mean residual, while clearly outperforming the fixed 1-step and GCC-PHAT baselines.

We also audited the CLI on two real public datasets. Figure 1 and Table 1 summarize matched 5-episode reruns on `dannyh147/so101_teleop` and `lerobot/aloha_static_screw_driver`. In both audits, the post-apply re-estimated lag falls to 0 in both `system_id` and `policy` modes, while the pre-apply lag-aware residual is preserved on the aligned output. These audits are small, but they show that the released CLI produces aligned outputs on which synchronous indexing becomes the effective optimum again.

The post-apply validation is best read as an internal workflow check. After materializing the aligned

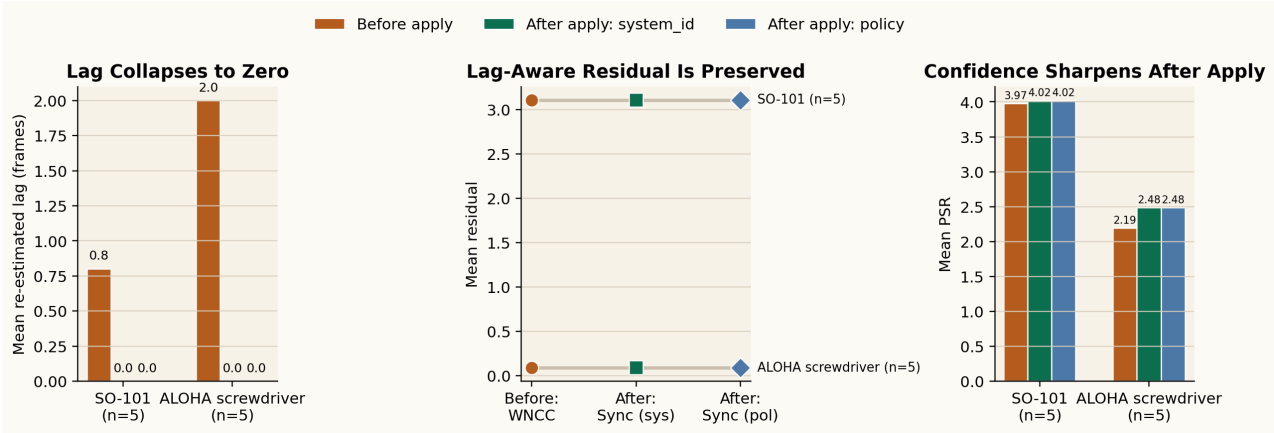

Figure 1: Focused external workflow audits of the released CLI on two real public datasets. Both panels use 5-episode subsets from `dannyh147/so101_teleop` and `lerobot/aloha_static_screw_driver`. Left: after `apply`, re-estimated lag collapses to 0 in both `system_id` and `policy` modes. Center: the pre-apply lag-aware residual is preserved on the aligned output; `Sync (sys)` denotes synchronous indexing on the aligned `system_id` output and `Sync (pol)` denotes synchronous indexing on the aligned `policy` output. Right: confidence sharpens slightly after apply, with PSR increasing on both datasets.

Table 1: Focused end-to-end CLI audits on two real Hugging Face LeRobot datasets. "Pre" values are computed on the raw datasets with `estimate` and `compare`; "post" values are computed by re-running those commands on the aligned outputs after `plan` and `apply`. In these two audits, the aggregate `system_id` and `policy` results are numerically identical after rounding, so one post-apply column suffices.

Dataset	Episodes	Pre lag	Post lag	Pre WNCC residual	Post sync residual	PSR pre → post
SO-101	5	0.800	0.000	3.110	3.110	3.974 → 4.018
ALOHA screwdriver	5	2.000	0.000	0.090	0.090	2.190 → 2.481

These are intentionally small workflow demonstrations, 5 episodes from `dannyh147/so101_teleop` and 5 episodes from `lerobot/aloha_static_screw_driver`, not a replacement for the full 388-episode public benchmark. The key pattern is that after `apply`, re-estimated lag collapses to zero while synchronous residual on the aligned output matches the original lag-aware residual computed on the same audited subset.

`system_id` parquet and re-running lag estimation, the mean re-estimated lag collapses from 4.194 frames to 0.014 frames, with median 0 frames. On the aligned output, synchronous indexing becomes effectively optimal: mean residual is 1.075 under synchronous indexing versus 1.079 under a fresh WNCC re-estimation, a negligible −0.188% change from trying to align the aligned data again. The same pattern appears in `policy` mode, where the re-estimated mean lag is again 0.014 frames and confidence diagnostics remain strong. This does not prove absolute timing accuracy, but it does show that the exported artifacts are self-consistent under the paper's own evaluation protocol.

These results support the main claim that explicit temporal calibration materially improves command-state consistency on the target SO-101 setting. They also explain why plain NCC is a strong comparator here: the calibration protocol excites one dominant joint at a time.

Table 2: Mean residual comparison for the command-state alignment baselines on the local calibration dataset.

Method	Residual	Improv. (%)
Synchronous indexing	4.007	0.000
Fixed 1-step shift	3.232	17.785
Plain NCC	1.075	64.409
GCC-PHAT	6.622	-34.181
LeCal WNCC	1.070	64.559

On this joint-isolation calibration set, dominant-joint plain NCC is a strong practical baseline and is nearly tied with LeCal WNCC, so aggregate residual is more informative than tie-broken winner counts.

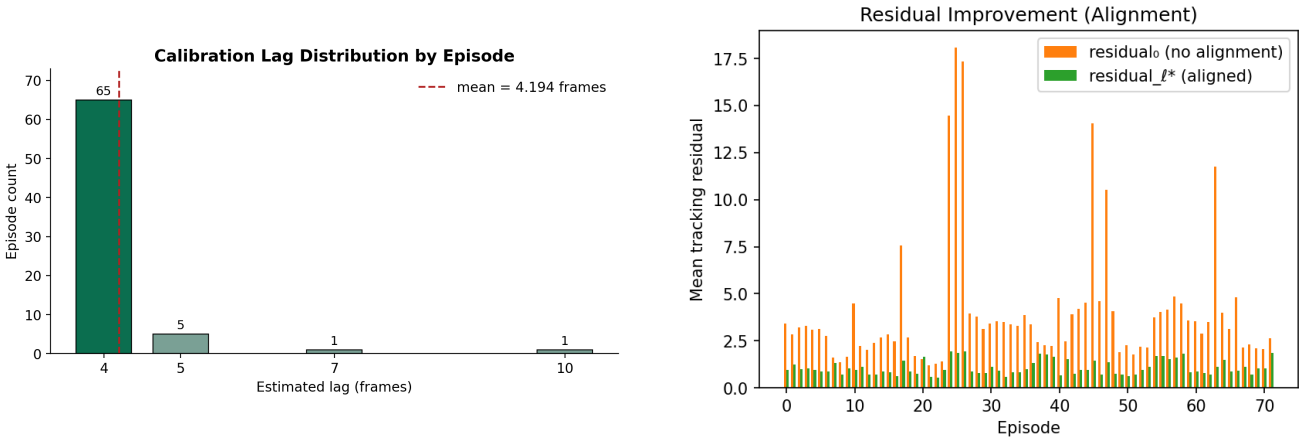

Figure 2: Source-backed visual summaries of the local calibration result. Left: the discrete episode-level lag distribution is sharply concentrated at 4 frames. Right: residuals drop substantially after applying the selected lag.

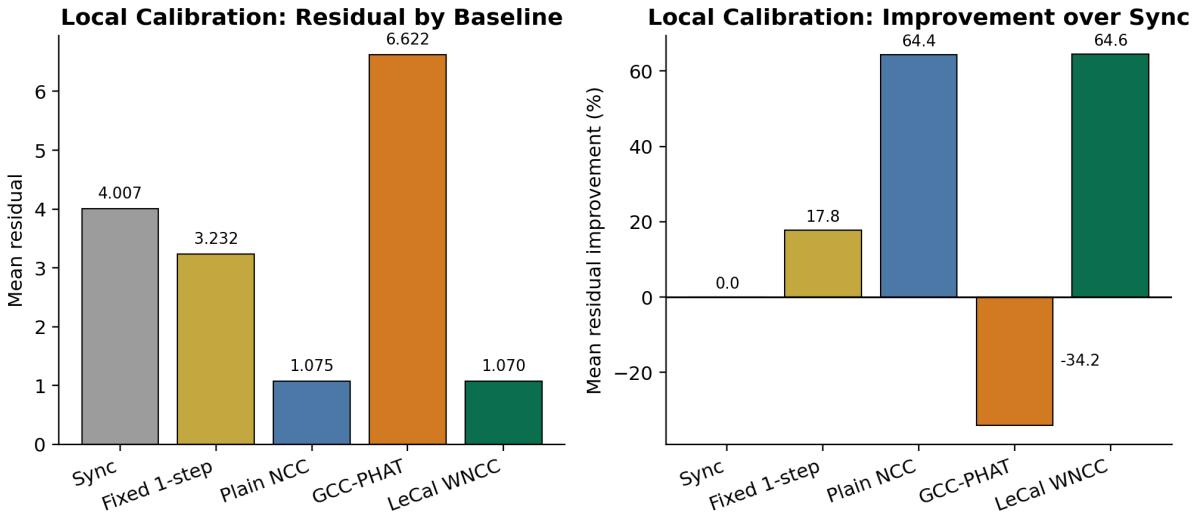

Figure 3: Compact local comparison across practical baselines. Plain NCC is a strong simple baseline on this joint-isolation dataset and is nearly tied with LeCal WNCC, while fixed 1-step and GCC-PHAT are clearly worse.

6.2 Full Public-Corpus Benchmark

The full public benchmark gives a more sober picture. Across the complete accessible public splits of eight LeRobot corpora, WNCC achieves a slightly lower weighted mean residual than plain NCC, 0.603 versus 0.605 over 388 episodes. At corpus level, WNCC is better on 5 of 8 datasets, tied on 1, and worse on 2. At episode level, the two methods choose the same integer lag on 277 of 388 episodes, so many downstream residuals are identical by construction. The gaps are therefore small. Even where WNCC wins, the gain over plain NCC is usually below 1% of NCC residual. To show where weighting actually changes the answer, we isolate a weighting-sensitive public subset: episodes for which no single joint carries more than 35% of the activity-weighted motion energy and the dominant-joint NCC heuristic chooses a different lag from WNCC. This leaves 109 episodes where estimator choice genuinely matters. In that subset, WNCC produces clearer gains on several corpora, but tape and thread-velcro remain counterexamples. The public result is therefore conditional rather than universal. The full per-corpus numeric breakdown is provided in the Zenodo companion package rather than repeated in the main draft [4].

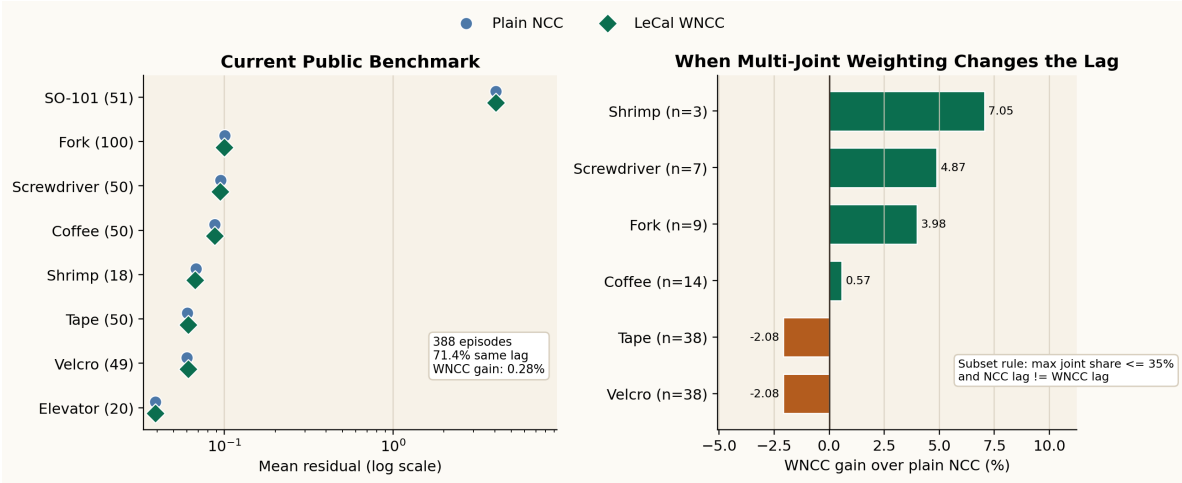

Figure 4: Two views of the public baseline story. Left: a corpus-level dumbbell plot for the full 388-episode benchmark shows that plain NCC and LeCal WNCC are close on most datasets, which is why the overall benchmark often looks nearly tied. Right: when attention is restricted to weighting-sensitive episodes, namely distributed-motion cases where the dominant-joint heuristic and WNCC choose different lags, multi-joint weighting produces clear gains on several corpora but not all of them.

6.3 Confidence and Consistency Diagnostics

Confidence and consistency diagnostics support the interpretation that the dominant local-dataset lag is not arbitrary. Across the 72 calibration episodes, mean peak correlation is 0.989, mean PSR is 1.484, mean latency-stability score is 0.757, and mean windowed lag sigma is 2.385 frames. These are heuristic internal diagnostics, not externally calibrated uncertainty estimates, but they are consistent with a stable dominant lag on most episodes.

Table 3: Confidence and consistency summary for the selected LeCal lag estimate on the local calibration dataset.

Metric	Mean	Min	Max
Peak correlation	0.989	0.833	0.999
PSR	1.484	1.280	2.463
Bootstrap sigma	1.927	0.000	8.886
Latency stability	0.757	0.116	2.008
Windowed lag sigma	2.385	0.000	8.698
Estimator disagreement	5.024	0.943	12.832

6.4 Camera Offset Reporting

LeCal also provides camera-to-proprioception timing analysis. In the released local subset, 12 gripper-only episodes are excluded because the camera is effectively static during isolated gripper actuation. The remaining 60 non-gripper episodes yield a small estimated camera offset, with mean 0.433 frames, median 0 frames, and range 0 to 1 frame. This supports camera-aware reporting and alignment planning, but not a stronger calibration claim than the action-state analysis.

Table 4: Camera-to-proprioception offset summary on the released local calibration subset used in this draft.

Statistic	Value
Total released local episodes	72
Camera-usable episodes	60
Skipped gripper-only videos	12
Mean offset (frames)	0.433
Median offset (frames)	0
Offset range (frames)	0 to 1
Global camera shift in plan	0 frames

7 Discussion

The results support a narrow but useful conclusion. LeCal clearly improves command-state alignment on the calibration dataset relative to practical baselines, but dominant-joint plain NCC is already very competitive on this joint-isolation protocol. The strongest case is therefore the full calibration workflow: robust estimation, diagnostics, camera-aware reporting, and exportable alignment artifacts rather than large estimator novelty. The post-apply validation supports that workflow framing, albeit only within the paper's internal evaluation protocol. When the exported artifacts are applied and re-audited, the measurable residual lag falls to essentially zero. The focused public re-audits point the same way. The public benchmark then sharpens the scope further: multi-joint weighting can help on distributed-motion episodes, but it is not a universal win. Because the current dataset does not include `observation.leader_state`, the strongest supported claim remains control-side to follower alignment using the recorded action stream as a surrogate.

8 Limitations

This paper has four main limitations. First, none of the evaluated datasets contains external ground-truth delay annotations, so evaluation is based on operational consistency rather than oracle timing error. Second, the post-apply re-estimation check is intentionally matched to the same family of alignment metrics and should be read as an internal consistency test. Third, although the public benchmark covers the full accessible splits of eight public corpora, those accessible splits are still smaller than the original raw corpora described in dataset cards. Fourth, the calibration dataset does not contain `observation.leader_state`, so the strongest supported claim is control-side-surrogate to follower alignment rather than full decomposition of leader, follower, and camera timing. These limitations define the scope of the paper rather than negate the main result. Stronger future validation would ideally include external timing references, synthetic delay injection, or manually annotated visual events.

9 Conclusion

LeCal provides a practical workflow for post hoc temporal calibration of LeRobot teleoperation datasets, together with a released implementation for downstream use [3]. On the released SO-101 calibration dataset on Hugging Face [2], explicit delay estimation materially improves command-state alignment relative to synchronous indexing, a fixed 1-step heuristic, and GCC-PHAT, while remaining competitive with a strong plain NCC baseline. Beyond lag estimation, LeCal adds diagnostics, camera-aware offset reporting, and reusable alignment artifacts. Camera analysis remains secondary, and the public benchmark suggests only modest robustness gains over plain NCC overall. With `observation.leader_state` and more independent timing checks, a final version could separate control-side, follower-side, and visual delays more directly.

References

[1] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, A. Zouitine, S. Palma, P. Kooijmans, M. Aractingi, M. Shukor, D. Aubakirova, M. Russi, F. Capuano, C. Pascal, J. Choghari, J. Moss, and T. Wolf, "LeRobot: State-of-the-art Machine Learning for Real-World Robotics in Pytorch," GitHub repository, 2024. [Online]. Available: https://github.com/huggingface/lerobot

[2] T. Chan, "lecal_calibration," Hugging Face dataset, 2026. [Online]. Available: https://huggingface.co/datasets/Toby0614/lecal_calibration

[3] T. Chan, "LeCal," GitHub repository, 2026. [Online]. Available: https://github.com/Toby0614/LeCal

[4] T. Chan, "LeCal paper companion data," Zenodo, 2026. doi: https://doi.org/10.5281/zenodo.19020018

[5] Hugging Face, "SO-101," LeRobot Documentation, 2026. [Online]. Available: https://huggingface.co/docs/lerobot/so101

[6] C. Bédard, I. Lütkebohle, and M. Dagenais, "ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6511–6518, 2022.

[7] C. Bédard, P.-Y. Lajoie, G. Beltrame, and M. Dagenais, "Message Flow Analysis with Complex Causal Links for Distributed ROS 2 Systems," *Robotics and Autonomous Systems*, vol. 161, p. 104361, 2023.

[8] C. H. Knapp and G. C. Carter, "The Generalized Correlation Method for Estimation of Time Delay," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 24, no. 4, pp. 320–327, 1976.

[9] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, S. Ibarz, P. Jang, C. Julian, D. Kalashnikov, S. Levine, Y. Lu, A. Luu, A. Parada, K. Pertsch, I. Rettinghouse, K. Reyes, T. Sermanet, N. Sievers, C. Wah, and T. Yu, "RT-1: Robotics Transformer for Real-World Control at Scale," arXiv:2212.06817, 2022.

[10] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," arXiv:2304.13705, 2023.

[11] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," arXiv:2303.04137, 2023.

[12] Open X-Embodiment Collaboration *et al.*, "Open X-Embodiment: Robotic Learning Datasets and RT-X Models," arXiv:2310.08864, 2023.

[13] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. Beltran Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Zhang, and S. Levine, "DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset," arXiv:2403.12945, 2024.

[14] BridgeData V2 Team, "BridgeData V2: A Dataset for Robot Learning at Scale," project page, 2023. [Online]. Available: https://bridgedata-v2.github.io/